Semantic subcategorization—Gildea and Palmer also used a semantic subcategorization frame where, in addition to the syntactic categories, the feature includes semantic role information.

While many researchers were constructing various features manually, Moschitti, Pighin, and Basili [132] tried a different approach. They used a tree kernel that identified and selected subtree patterns from a large number of automatically generated patterns to capture the tree context. Unfortunately, for this application, the performance was somewhat inferior to the manually selected features. It is possible that for some other machine learning problem where handcrafting of features is not as intuitive, this technique could prove valuable.

Dependency Trees One issue in the formulations so far is that the performance of a system depends on the exact span of the arguments annotated according to the constituents in the Penn Treebank. Labels are scored as correct only if they match the PropBank annotation exactly; both the bracketing and the label must match. Because PropBank and most syntactic parsers are developed on the Penn Treebank corpus and are therefore based on the same syntactic structures, they would be expected to match the PropBank labeling better than the other representations. But does a better score here imply that the output is more usable for any applications built on the role labels? It may often be the case that the specific bracketing is not really important; rather, the critical information is the relation of the argument head word to the predicate. Scoring the output of the algorithm using this strategy gives a much higher performance with an F-score of about 85 (vs. 79).

Hacioglu [133] formulated the problem of semantic role labeling on a dependency tree by converting the Penn Treebank trees to a dependency representation using a script by Hwa, Lopez, and Diab [134] and creating a dependency structure labeled with PropBank arguments.

The performance on this system seemed to be about 5 F-score points better than the one trained on the phrase structure trees. One possible shortcoming of this and other approaches is that all the parsers are trained on the same Penn Treebank, and when evaluated on sources other than WSJ, seem to degrade in performance. Pradhan et al. [129] experimented to find how well a rule-based dependency parser might fare. Minipar [135, 136] is the rule-based dependency parser that was used. It outputs dependencies between a word called **head** and another called **modifier**. Each word can modify at most one word. The dependency relationships form a dependency tree. The set of words under each node in Minipar's dependency tree form a contiguous segment in the original sentence and correspond to the constituent in a constituent tree. Figure 4–13 shows how the arguments of the predicate *kick* map to the nodes in a phrase structure grammar tree as well as the nodes in a Minipar parse tree. The nodes that represent head words of constituents are the targets of classification. They used the same features as Hacioglu [133] (see Table 4–4).

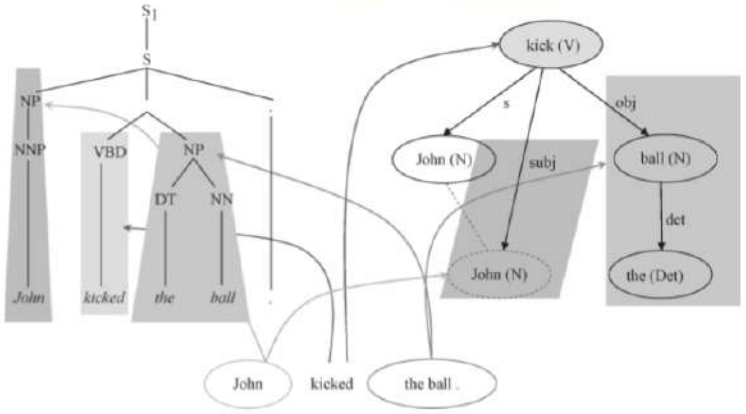

Figure 4–13. New architecture

Table 4–4. Features used in the baseline system using Minipar parses

Head word	The word representing the node in the dependency tree.
Head word POS	Part of speech of the head word.
POS path	The path from the predicate to the head word through the dependency tree connecting the part of speech of each node in the tree.
Dependency path	Each word that is connected to the head word has a dependency relationship to the word. These are represented as labels on the arc between the words. This feature comprises the dependencies along the path that connects two words.
Voice	Voice of the predicate.
Position	Whether the node is before or after the predicate.

Minipar performance on the PropBank corpus is substantially worse than the Charniakbased system (47.2 if computed using the strict span criteria). This is expected, as Minipar is not designed to produce constituents that exactly match the constituent segmentation used in the Penn Treebank. In experiments reported by Hacioglu [133], a mismatch of about 8% was introduced in the transformation from Treebank trees to dependency trees. Using an errorful automatically generated tree, a still higher mismatch would be expected. In the case of the CCG parses, as reported by Gildea and Hockenmaier [128], the mismatch was about 23%. A more realistic way to score the performance is to score tags assigned to head words of constituents, rather than considering the exact boundaries of the constituents. This score turns out to be an F-score of about 61.7, which is much better and provides orthogonal benefits. These results are a compelling argument for the integration of dependency trees with phrase structure predicate-argument structure.

Since then, there was significant work done on dependency parsing, which led to a series of experiments in this vein. Two Computational Natural Language Learning (CoNLL) shared tasks [137, 138] were held to further research ways to combine dependency parsing and semantic role labeling. These included the use of a richer syntactic dependency representation by Johansson and Nugues [139], which considers tree gaps and traces, and mapping PropBank predicates and arguments to that representation.

Base Phrase Chunks A common question is, How much does the full syntactic representation help the task of semantic role labeling? In other words, how important is it to create a full syntactic tree before classifying the arguments of a predicate? A chunk representation can be faster and more robust to phrase reordering as happens in speech data. Gildea and Palmer [131] explored this question using a chunk-based approach and concluded that syntactic parsing helps fill a big gap. Hacioglu [124] further experimented with a chunk-based semantic labeling method and reached a somewhat more optimistic conclusion. Punyakanok, Roth, and Yih [140] also reported experiments in chunking. Generally, chunk-based systems classify each base phrase as the B(eginning) of a semantic role, I(nside) a semantic role, or O(utside) any semantic role (i.e., null). This is referred to as an IOB representation [141]. This system uses SVM classifiers to first chunk input text into flat chunks or base phrases, each labeled with a syntactic tag. A second SVM is trained to assign semantic labels to the chunks. Figure 4–14 shows a schematic of the chunking process. Table 4–5 lists the features used by the semantic chunker.

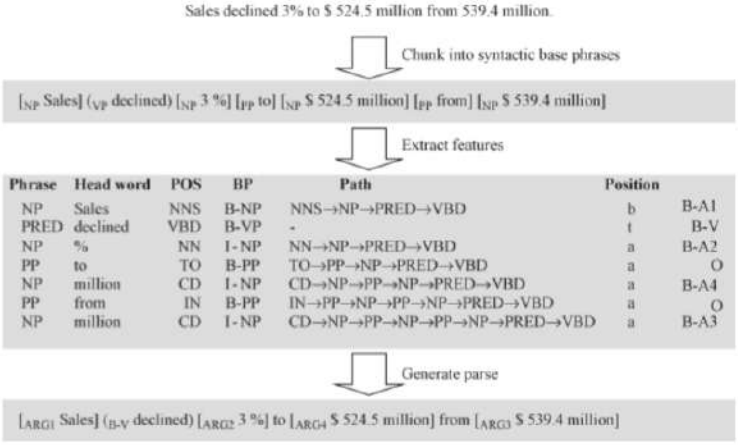

Figure 4–14. Semantic chunker

Table 4–5. Features used by chunk-based classifier

Words	Words in the chunk.
Predicate lemma	The predicate lemma.
POS tags	Part of speech of the words in the chunk.
BP positions	The position of a token in a base phrase (BP) using the IOB2 representation (e.g., B-NP, I-NP, O).
Clause tags	The tags that mark token positions in a sentence with respect to clauses.
Named entities	The IOB tags of named entities.
Token position	The position of the phrase with respect to the predicate. It has three values: "before," "after," and "-" (for the predicate).
Path	Defines a flat path between the token and the predicate.
Clause bracket patterns	
Clause position	A binary feature that identifies whether the token is inside or outside the clause containing the predicate.
Headword suffixes	Suffixes of head words of length 2, 3, and 4.
Distance	Distance of the token from the predicate as a number of base phrases and the distance as the number of VP chunks.
Length	The number of words in a token.
Predicate POS tag	The part of speech category of the predicate.
Predicate frequency	Frequent or rare using a threshold of 3.
Predicate BP context	The chain of BPs centered at the predicate within a window of size $-2/+2$.
Predicate POS context	POS tags of words immediately preceding and following the predicate.
Predicate-argument frames	Left and right core argument patterns around the predicate.
Number of predicates	This is the number of predicates in the sentence.

For each token (base phrase) to be tagged, a set of features is created from a fixedsize context that surrounds each token. In addition to the above features, the chunker uses previous semantic tags that have already been assigned to the tokens contained in the linguistic context. A five-token sliding window is used for the context. The performance on the task of identification and classification was an F-score of about 70.

Classification Paradigms

In the previous section, we looked at sentence-level structural representations that have been used to tackle the problem of semantic role labeling and the features in each of those representations that were identified to capture those characteristics that would help train a model to identify them automatically. In this section, we focus on the ways in which machine learning has been brought to bear on the problem. The approaches range widely in their complexity. The simplest approaches are those that view semantic role labeling as a pure classification problem, where each argument of a predicate may be classified

independently of the others. Other researchers have adopted this basic paradigm but added a simple postprocessor that removes implausible analysis, such as when two arguments overlap. A few more complicated approaches augment the postprocessing step to use argumentspecific language models or frame-element group statistics. These postprocessors tend to take care of a significant portion of the problems introduced by the original independence assumption.

There have also been more sophisticated approaches to performing joint decoding of all the arguments, trying to capture the arguments' interdependence. Unfortunately, these sophisticated approaches have so far yielded only slight gains, partly because the performance with a pure classifier followed by a simple postprocessor is already quite high. In this section, rather than provide detailed descriptions of all previous approaches, we concentrate on a current high-performing approach that effectively utilizes multiple knowledge sources and uses a combined architecture that degrades gracefully when run on out-of-genre text.

To begin with, Gildea and Jurafsky's [113] variation of the SRL algorithm involves two steps. In the first step, the system calculates maximum likelihood probabilities that the constituent is an argument, based on these two features, P (argument|Path, Predicate) and P (argument|Head, Predicate), and interpolates them to generate the probability that the constituent under consideration represents an argument. In the second step, it assigns each constituent that has a nonzero probability of being an argument a normalized probability calculated by interpolating distributions conditioned on various sets of features, and selects the most probable argument sequence. Some of the distributions they used are shown in Table 4–6.

Table 4–6. Distributions used for semantic argument classification, calculated from the features extracted from a Charniak parse

Distributions
P(argument|Predicate)
P(argument|Phrase Type, Predicate)
P(argument|Phrase Type, Position, Voice)
P(argument|Phrase Type, Position, Voice, Predicate)
P(argument|Phrase Type, Path, Predicate)
P(argument|Phrase Type, Path, Predicate, Subcategorization)
P(argument|Head Word)
P(argument|Head Word, Predicate)
P(argument|Head Word, Phrase Type, Predicate)

Surdeanu et al. [118] used a decision tree classifier algorithm, C5 [142, 143] on the same features as Gildea and Jurafsky [113]. The built-in boosting capabilities of this classifier gave a slight improvement in performance. Chen and Rambow [130] also report results using a decision tree classifier, C4.5 [142]. Fleischman and Hovy [144] report results on the FrameNet corpus using a maximum entropy framework. Pradhan et al. [120] used SVM for the same and got even better performance on the PropBank corpus. Nonetheless, the difference between a maximum entropy classifier and SVM turned out to be small.

Pradhan et al. [120] report results using the same set of features on the same data and compare the classifiers with each other. Gildea and Palmer's [131] system estimates the posterior probabilities using several different feature sets and interpolates the estimates—exactly like the Gildea and Jurafsky [113] system—whereas Surdeanu et al. [118] use a decision tree classifier. Table 4–7 shows compares the three systems for the task of argument classification.

Table 4–7. Argument classification using same features but different classifiers

Classifier	Accuracy (%)
SVM (Pradhan et al.) [120]	88
Decision Tree (Surdeanu et al.) [118]	79
Gildea and Palmer [131]	77

For our discussion, we use TinySVM[10] along with YamCha[11] as the SVM training and test software [145, 146]. The SVM parameters, such as the type of kernel, and the values of various parameters was empirically determined using the development set. A polynomial kernel of degree 2 was selected with the cost per unit violation of the margin $C = 1$ and tolerance of the termination criterion $e = 0.001$.

SVMs perform well on text classification tasks where data are represented in a highdimensional space using sparse feature vectors [147, 148]. Inspired by the success of using SVMs for tagging syntactic chunks [145], Pradhan et al. [149, 126, 120] formulated the semantic role labeling problem as a multiclass classification problem using SVMs.

SVMs are inherently binary classifiers, but multiclass problems can be reduced to a number of binary-class problems using either the pairwise approach or the one versus all (OVA) approach [150]. For an N class problem, in the pairwise approach, a binary classifier is trained for each pair of the possible $\frac{N(N-1)}{2}$ class pairs, whereas, in the OVA approach, N binary classifiers are trained to discriminate each class from a metaclass created by combining the rest of the classes. Comparing these two approaches, there is a trade-off between the number of classifiers to be trained and the data used to train each classifier. Although some experiments have reported that the pairwise approach outperforms the OVA approach [151], Pradhan et al.'s [120] initial experiments show better performance for OVA. Therefore, they chose the OVA approach.

SVM outputs the distance of a feature vector from the maximum margin hyperplane. To facilitate probabilistic thresholding and generate an n-best hypotheses lattice, they convert the distances to probabilities by fitting a sigmoid to the scores, as described in Platt [152].

The system can be viewed as comprising two stages: the training stage and the testing stage. We first discuss how the SVM is trained for this task. Because the training time taken

by SVMs scales exponentially with the number of examples, and about 90% of the nodes in a syntactic tree have null argument labels, it is efficient to divide the training process into two stages:

1. Filter out the nodes that have a very high probability of being null. A binary null/nonnull classifier is trained on the entire dataset. Fit a sigmoid function to the raw scores to convert the scores to probabilities, as described by Platt [152]. All the training examples are run through this classifier, and the respective scores for null and nonnull assignments are converted to probabilities using the sigmoid function. Nodes that are most likely null (probability >0.90) are pruned from the training set. This reduces the number of null nodes by about 90% and the total number of nodes by about 80%. This is accompanied by a negligible (about 1%) pruning of nodes that are non-null.
2. The remaining training data are used to train OVA classifiers for all the classes along with a null class.

With this strategy, only one classifier (null or non-null) has to be trained on all of the data. The remaining OVA classifiers are trained on the nodes passed by the filter (approximately 20% of the total), resulting in a considerable savings in training time.

In the testing stage, all the nodes are classified directly as null or one of the arguments using the classifier trained in step 2. They observe a slight decrease in recall if we filter the test examples using a null/non-null OVA classifier in a first pass, as we do in the training process. This small performance gain is obtained at little or no cost of computation because SVMs are very fast in the testing phase. Pseudocode for the testing algorithm is shown earlier in Figure 4–3. A variation in this strategy would be to filter all the examples that are null in the first pruning stage instead of just pruning out the high-probability ones. This

strategy, however, has a statistically significant performance degradation associated with it.

On gold-standard Treebank parses, the performance of such a system on the combined task of argument identification and classification is in the low 90s, whereas on automatically generated parses, the performance tends to be in the high 70s.

Overcoming the Independence Assumption

As mentioned earlier, various postprocessing stages have been proposed to overcome the limitations of treating semantic role labeling as a series of independent argument classification steps. We now look at some of these strategies.

Disallowing Overlaps Since each constituent is classified independently of the other, it is possible that two constituents that overlap get assigned an argument type. Because we are dealing with parse tree, nodes overlapping in words always have an ancestor-descendant relationship, and therefore the overlaps are restricted to subsumptions only as shown in Example 4–1.

Example 4–1: But $[_{ARG0}$ nobody$]$ $[_{predicate}$ knows $]$ $[_{ARG1}$ at what level $[_{ARG1}$ the futures and stocks will open today$]]$

This is a problem because overlapping arguments were not allowed in PropBank (or, more specifically, any two arguments of a verb predicate even in FrameNet cannot overlap). One way to deal with this issue is to choose among overlapping constituents by retaining the one for which the SVM has the highest confidence based on the classification probabilities and labeling the others Null. The probabilities obtained by applying the sigmoid function to the raw SVM scores are used as the measure of confidence.

Argument Sequence Information Another way makes use of the fact that a predicate is likely to instantiate a certain set of argument types as done by Gildea and

Jurafsky (2002) [113] to improve the performance of their statistical argument tagger. A similar but more principled approach involves imposing additional constraints in which argument ordering information is retained and the predicate is considered as an argument and is part of the sequence. This can be achieved by training a trigram language model on the argument sequences by first converting the raw SVM scores to probabilities, as described earlier. Then, for each sentence being parsed, an argument lattice is generated using the n-best hypotheses for each node in the syntax tree. Then a Viterbi search is performed through the lattice using the probabilities assigned by the sigmoid as the observation probabilities, along with the language model probabilities, to find the maximum likelihood path through the lattice such that each node is assigned a value belonging to the PropBank arguments or null.

The search is constrained in such a way that no two non-null nodes overlap. To simplify the search, Pradhan et al. [120] allowed only null assignments to nodes having a null likelihood above a threshold. While training the language model, we can use the actual predicate to estimate the transition probabilities in and out of the predicate, or we can perform a joint estimation over all the predicates. Pradhan et al. found that there was an improvement in the core arguments accuracy on the combined task of identifying and assigning semantic arguments, whereas the accuracy of the adjunctive arguments slightly deteriorated. This seems logical considering that the adjunctive arguments have looser constraints on their ordering and even their quantity. It is therefore beneficial to use this strategy only for the core arguments. Some other strategies have been used to incorporate argument context information. Toutanova, Haghighi, and Manning [153] report performance on using a more global model to predict semantic roles for a given predicate using log-linear models, whereas Punyakanok et al. [154] use an integer linear programming–based inference

framework to improve the performance of semantic role labeling. Both show small gains over the Viterbi approach.

Feature Performance

Not all features are equally useful in each task. Some features add more noise than information in one context than in another, and features can vary in efficacy depending on the classification paradigm in which they are used. Table 4–8 shows the effect each feature has on the argument classification and argument identification tasks when added individually to the baseline. Addition of named entities to the null/non-null classifier degraded its performance in this particular configuration of classifier and features. This effect can be attributed to a combination of two things: (i) a significant number of constituents contain named entities but are not arguments of a predicate (the parent of an argument node also contains the same named entity), resulting in a noisy feature for null/non-null classification; and (ii) SVMs don't seem to handle irrelevant features very well [155]. When this feature was solely used in the task of classifying constituents known to represent arguments, using features extracted from Treebank parses, the overall classification accuracy increased from 87.9% to 88.1%, whereas adding head word POS as a feature significantly improves both the argument classification and the argument identification tasks.

Table 4–8. Effect of each feature on the argument classification task and argument identification task when added to the baseline system. An asterisx indicates that the improvement is statistically significant

FEATURES	ARGUMENT CLASSIFICATION	ARGUMENT IDENTIFICATION		
	A	P	R	F_1
Baseline [120]	87.9	93.7	88.9	91.3
+ Named entities	**88.1**	93.3	88.9	91.0
+ Head POS	* **88.6**	94.4	90.1	* **92.2**
+ Verb cluster	**88.1**	94.1	89.0	**91.5**
+ Partial path	**88.2**	93.3	88.9	91.1
+ Verb sense	**88.1**	93.7	89.5	**91.5**
+ Noun head PP (only POS)	* **88.6**	94.4	90.0	* **92.2**
+ Noun head PP (only head)	* **89.8**	94.0	89.4	**91.7**
+ Noun head PP (both)	* **89.9**	94.7	90.5	* **92.6**
+ First word in constituent	* **89.0**	94.4	91.1	* **92.7**
+ Last word in constituent	* **89.4**	93.8	89.4	**91.6**
+ First POS in constituent	**88.4**	94.4	90.6	* **92.5**
+ Last POS in constituent	**88.3**	93.6	89.1	91.3
+ Ordinal const. pos. concat.	87.7	93.7	89.2	**91.4**
+ Const. tree distance	**88.0**	93.7	89.5	**91.5**
+ Parent constituent	**87.9**	94.2	90.2	* **92.2**
+ Parent head	85.8	94.2	90.5	* **92.3**
+ Parent head POS	* **88.5**	94.3	90.3	* **92.3**
+ Right sibling constituent	**87.9**	94.0	89.9	**91.9**
+ Right sibling head	**87.9**	94.4	89.9	* **92.1**
+ Right sibling head POS	**88.1**	94.1	89.9	**92.0**
+ Left sibling constituent	* **88.6**	93.6	89.6	**91.6**
+ Left sibling head	86.9	93.9	86.1	89.9
+ Left sibling head POS	* **88.8**	93.5	89.3	**91.4**
+ Temporal cue words	* **88.6**	-	-	-
+ Dynamic class context	**88.4**	-	-	-

Feature Salience

In analyzing the performance of the system, it is useful to estimate the relative contribution of the various feature sets used. Table 4–9 shows the argument classification accuracies for combinations of features on the training and test set for all PropBank arguments, using Treebank parses.

Table 4–9. Performance of various feature combinations on the task of argument classification

FEATURES	ACCURACY
All features [120]	91.0
All except Path	90.8
All except Phrase Type	90.8
All except HW and HW-POS	90.7
All except All Phrases	*83.6
All except Predicate	*82.4
All except HW and FW and LW info.	*75.1
Only Path and Predicate	74.4
Only Path and Phrase Type	47.2
Only Head Word	37.7
Only Path	28.0

In the upper part of Table 4–9 we see the degradation in performance by leaving out one feature at a time. The features are arranged in the order of increasing salience. Removing all head word-related information has the most detrimental effect on performance. The lower part of the table shows the performance of some feature combinations by themselves. Table 4–10 shows the feature salience on the task of argument identification. As opposed to the argument classification task, where removing the path has the least effect on performance, on the task of argument identification, removing the path causes the convergence in SVM training to be very slow and has the most detrimental effect on performance.

Table 4–10. Performance of various feature combinations on the task of argument identification

FEATURES	P	R	F_1
All features [120]	95.2	92.5	93.8
All except HW	95.1	92.3	93.7
All except Predicate	94.5	91.9	93.2
All except HW and FW and LW info.	91.8	88.5	*90.1
All except Path and Partial Path	88.4	88.9	*88.6
Only Path and HW	88.5	84.3	86.3
Only Path and Predicate	89.3	81.2	85.1

Feature Selection

The fact that adding the named entity features to the null/non-null classifier had a detrimental effect on the performance of the argument identification task, while the same feature set showed significant improvement to the argument classification task, indicates that a feature selection strategy would be beneficial. One strategy would be to perform a leave-one-out experiment whereby one feature is left out of the full set of features at a time, and depending on the degradation in performance, it is either kept of pruned out. This is a very naïve feature-selection

strategy that assumes the features are independent of each other. We could use a more complicated feature-selection strategy instead. One downside of feature selection based on each argument type in the SVM paradigm is that SVMs output distances, not probabilities. These distances may not be comparable across classifiers, especially if different features are used to train each binary classifier. A solution is to use the algorithm described by Platt [152] to convert the SVM scores into probabilities by fitting to a sigmoid. Foster and Stine [156] show that the pool-adjacent-violators (PAV) algorithm [157] provides a better method for converting raw classifier scores to probabilities when Platt's algorithm fails. The probabilities resulting from either conversion may not be properly calibrated, in which case the probabilities can be binned, and a warping function can be trained to calibrate them.

Size of Training Data

One important concern in any supervised learning method is the amount of training examples required for decent performance of a classifier. To check the behavior of this learning problem, Pradhan et al. [129] trained the classifiers on varying amounts of training data. The resulting plots are shown in Figure 4–15. The first curve from the top indicates the change in F_1 score on the task of argument identification alone. The third curve indicates the F_1 score on the combined task of argument identification and classification. It can be seen that after about 10,000 examples, the performance starts to plateau, which indicates that simply tagging more data might not be a good strategy. A better strategy is to tag only appropriate new data. Also, the fact that the first and third curves—the first being the F-score on the task of argument identification task and the third being the F-score on the combined task of identification and classification—run almost parallel to each other tells us that a constant loss occurs due to classification errors throughout the data

range. One way to bridge this gap could be to identify better features.

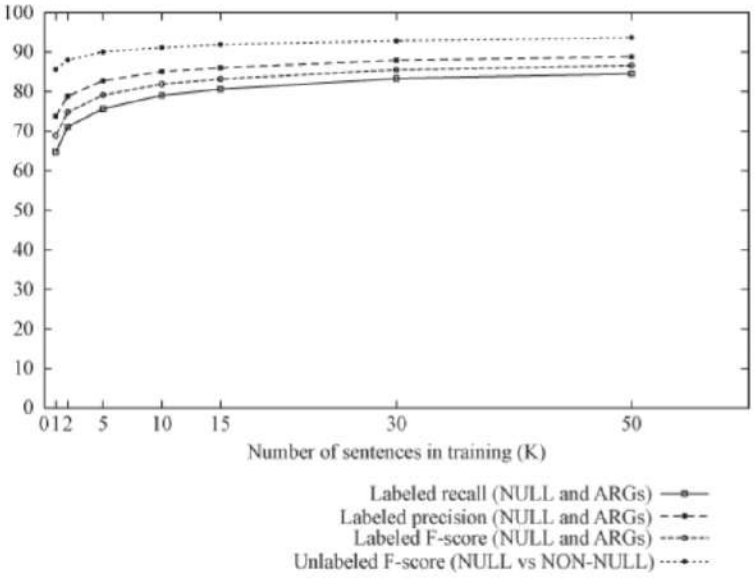

Figure 4–15. Learning curve for the tasks of identifying and classifying arguments using Treebank parses

Overcoming Parsing Errors

After performing a detailed error analysis, Pradhan et al. [129] found that the identification problem poses a significant bottleneck to improving overall system performance. The baseline system's accuracy on the task of labeling nodes known to represent semantic arguments is 90%. On the other hand, the system's performance on the identification task is quite a bit lower, achieving only 80% recall with 86% precision. The two sources of these identification errors are failures by the system to identify all and only those constituents that correspond to semantic roles, *when those constituents are present in the syntactic analysis*, and failures by the syntactic analyzer to provide the constituents that align with correct arguments. Classification performance using Charniak parses is about 3 F-score points worse than when using treebank parses. On the other hand, argument identification performance using Charniak parses is about 12.7 F-score points worse.

Half of these errors—about 7 points—are due to missing constituents, and the other half—about 6 points—are due to misclassifications.

This severe degradation in argument identification performance for automatic parses was the motivation for examining two techniques for improving argument identification: combining parses from different syntactic representations and using *n*-best parses or a parse forest in the same representation.

Multiple Views Pradhan et al. [129] report on experiments that address the problem of arguments missing from a given syntactic analysis. They investigate ways to combine hypotheses generated from semantic role taggers trained using different syntactic views: one trained using the Charniak parser [158]; another on a rule-based dependency parser, Minipar [135]; and a third based on a flat, shallow syntactic chunk representation [159]. They showed that these three views complement each other to improve performance.

Some of the systems we have discussed use features based on syntactic constituents produced by a syntactic parser [149, 126], and others use only a flat syntactic representation produced by a syntactic chunker [160, 159, 124]. The latter approach lacks the information provided by the hierarchical syntactic structure, and the former imposes the limitation that the possible candidate roles should be one of the nodes already present in the syntax tree. Although the chunk-based systems are very efficient and robust, the systems that use features based on full syntactic parses are generally more accurate. Analysis of the source of errors for the parse constituent–based systems showed that incorrect parses were a major source of error. The syntactic parser often did not produce any constituent that corresponded to the correct segmentation for the semantic argument. Pradhan et al. [129] report on a first attempt to overcome this problem by combining semantic role labels produced from different

syntactic parses. The hypothesis is that the syntactic parsers will make different errors, and combining their outputs will be an improvement over any one system. This initial attempt used features from the Charniak parser, the Minipar parser, and a chunk-based parser. It did show some improvement from the combination, but the method for combining the information was heuristic and suboptimal. The researchers proposed an improved framework for combining information from different syntactic views. The goal was to preserve the robustness and flexibility of the segmentation of the phrase-based chunker but to take advantage of features from full syntactic parses. They also wanted to combine features from different syntactic parses to gain additional robustness. To this end, they used features generated from the Charniak parser and the Collins parser. The main contribution of combining both the Minipar-based and the Charniak-based semantic role labeler was significantly improved performance on ARG1 in addition to slight improvements to some other arguments. See Figure 4–16.

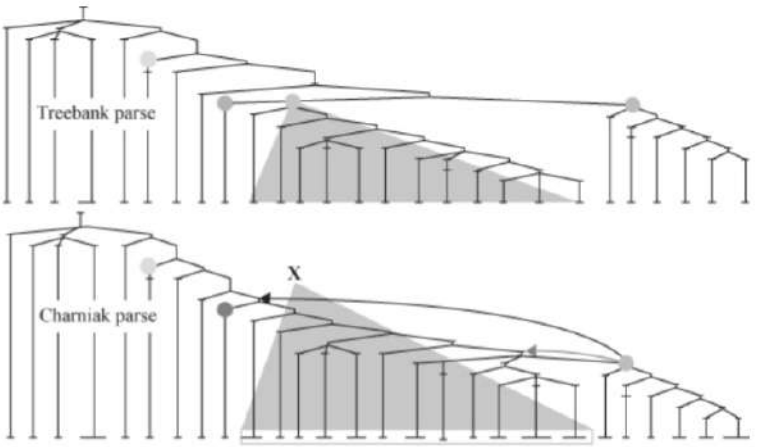

Figure 4–16. Argument deletions owing to parse error

The semantic parses were combined as follows. Scores for arguments were converted to calibrated probabilities, and arguments with scores below a threshold value were deleted. Separate thresholds were used for each semantic role labeler.

For the remaining arguments, if any set of arguments overlapped, the least probable among them were removed until no overlaps remained. In the chunk-based system, an argument could consist of a sequence of chunks. The probability assigned to the BEGIN tag of an argument was used as the probability of the sequence of chunks forming an argument.

The general framework is to train separate semantic role labeling systems for each of the parse tree views, and then to use the role arguments output by these systems as additional features in a semantic role classifier using a flat syntactic view. The constituent-based classifiers walk a syntactic parse tree and classify each node as null (no role) or as one of the set of semantic roles. As we saw in Section 4.5.2, chunk-based systems classify each base phrase using the IOB representation. The constituent level roles are mapped to the IOB representation used by the chunker. The IOB tags are then used as features for a separate base-phrase semantic role labeler (chunker), in addition to the standard set of features used by the chunker. An n-fold cross-validation paradigm is used to train the constituent-based role classifiers and the chunk-based classifier. See Figure 4–17.

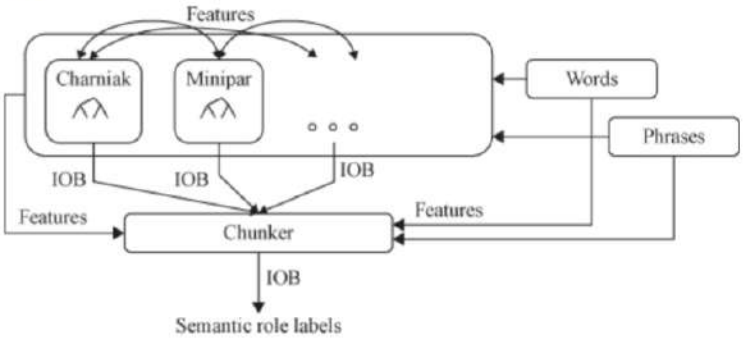

Figure 4–17. New architecture

The chunking system for combining all features is trained using a fourfold paradigm. In each fold, separate SVM classifiers are trained using 75% of the training data. The other 25% of the training data are then labeled by each of

the systems. Iterating this process four times creates the training set for the chunker. After the chunker is trained, the PSG and Minipar-based semantic labelers are retrained using all of the training data. Once the retraining is complete, SVMs are trained for begin (B) and inside (I) classes of all arguments and an outside (O) class. One particular advantage of this architecture, as depicted in Figure 4–18, is that the final segmentation does not necessarily have to adhere to one of the input segmentations. Depending on the information provided in terms of features, the classifier can generate a new, better segmentation.

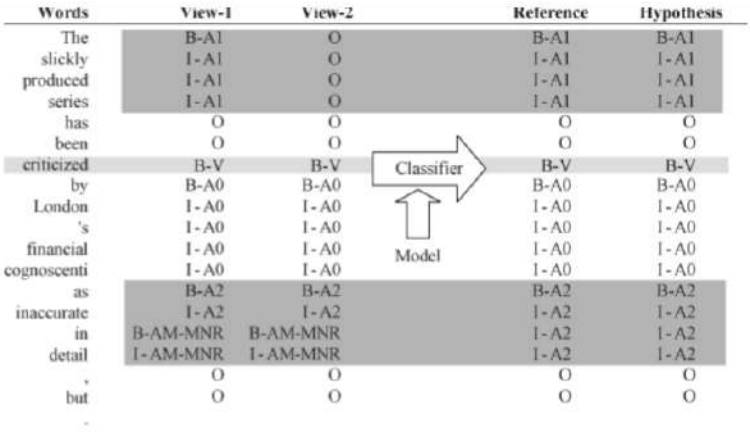

Figure 4–18. Example classification using the new architecture

This is just one way of combining various views. Another combination strategy by Surdeanu et al. [161] also shows improvement over using a single view.

Broader Search Another approach is to broaden the search by selecting constituents in n-best parses [153, 154] or using

a packed forest representation [162], which more efficiently represents variations over much larger n. Using a parse forest shows an absolute improvement of 1.2 points over single best parses and 0.5 points over n-best parses.

Noun Arguments

Until now we have only dealt with the task of identification and classification of arguments of verb predicates in a sentence. To generate a sentence-level semantic representation, it is necessary to identify arguments of other possible predicates in a sentence, such as nominal predicates, adjectival predicates, and prepositional predicates. This chapter discusses the task of semantic role labeling as applied to a class of nominal predicates, or nominalizations. A simple linguistic definition of nominalization is "the process of converting a verb into an abstract noun." Take, for example, the two sentence pairs in Figure 4–19, where the second sentence in each pair is a nominalized version of the first. One important thing to note is that the verbs in the nominalized sentences are *make* and *took* respectively. A semantic analyzer that parses these sentences for the arguments of these verbs will miss the events—*complaining* and *walking*—that are central to understanding the meaning of the sentences and that are represented by the nominal predicates *complain* and *walk* respectively.

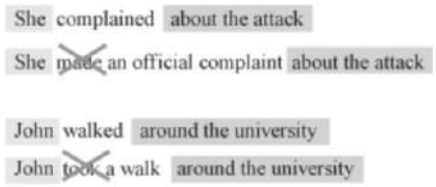

Figure 4–19. Example of nominalization

Of the vast literature on automatic semantic role labeling, very little deals with assigning the semantic role properties of nouns. Of what does, most deal with nominalizations. However, because of the lack of a corpus annotated with nominal predicates and their arguments, there have been

no investigations into applying statistical algorithms that can automatically identify and label the arguments of nouns. To our knowledge, the only works that come close are the rule-based system described by Hull and Gomez [163] and the work of Lapata [164] on interpreting the relationship between the head of a nominalized compound and its modifier noun. With the availability of hand-labeled argument information for nominal predicates from the FrameNet project, an investigation into the feasibility of automatically identifying nominal predicates using this data can be performed.

In this section, we look at the adaptation to nouns of features that were originally derived for verbs; most of these adaptations are quite straightforward. We also investigate how well these transformed features identify the semantic properties of noun arguments. In other words, how good are these features at identifying and classifying nominal arguments? Furthermore, can any new sets of features specific to nominalizations be added to good effect?

Following are some new features that were identified by Pradhan et al. [165] along with a justification. Some of these features don't exist for some constituents. In those cases, the respective feature values are set to UNK. Almost all the new features that we used for the verb predicates, except the CCG features, were added to the baseline system.

Intervening verb features—Support verbs play an important role in realizing the arguments of nominal predicates. Three classes of intervening verbs were used: (i) verbs of being, (ii) light verbs (a small set of verbs such as *make, take, have*), and (iii) other verbs with part of speech starting with the string VB. Three features were added for each: (i) a binary feature indicating the presence of the verb between the predicate and the constituent, (ii) the actual word as a feature, and (iii) the path through the tree from the constituent to the verb. The following example

illustrates the intuition behind these intervening verb features:

[$_{Speaker}$ Leapor] *makes* general [$_{Predicate}$ assertions] [$_{Topic}$ about marriage]

Predicate NP expansion rule—This is the noun equivalent of the verb subcategorization feature used by Gildea and Jurafsky [113]. It represents the expansion rule instantiated by the syntactic parser, for the lowermost NP in the tree, encompassing the predicate. This feature tends to cluster noun phrases with a similar internal structure and thus helps find argument modifiers.

Is predicate plural—This binary feature indicates whether the predicate is singular or plural, as these tend to have different argument selection properties.

Genitives in constituent—This is a binary feature that is true if there is a genitive word (one with the part of speech POS, PRP, PRP$, or WP$) in the constituent, as these tend to be subject/object markers for nominal arguments. The following example helps clarify this notion:

[$_{Speaker}$ Burma 's] [$_{Phenomenon}$ oil] [$_{Predicate}$ search] hits virgin forests

Verb dominating predicate—The head word of the first VP ancestor of the predicate.

More recently, Jiang and Ng [166] use additional features in a maximum entropy framework to perform argument tagging on the NomBank corpus. Also, NomBank arguments have been added to an integrated syntax-semantic dependency representation in the recent CoNLL evaluations [137, 138].

Multilingual Issues

Because early research on semantic role labeling was performed on English corpora, various core features and learning mechanisms were explored specifically for English.

Apparently, most of the core features for English translate well for other languages [167, 168]. Some special cases of language-specific features turned out to be important for a specific language but also improved performance for English systems; for example, the predicate frame feature that was introduced by Xue and Palmer [127] for Chinese also shows some features improvement in English. Some features are so language specific that they have no parallels in English, and those usually are unique to a particular language; for example, Chinese requires a much more complex word segmentation process—something that can be performed quite accurately in English using very simple rules. Therefore, special word segmentation models have to be trained in the case of Chinese before parsing or semantic role labeling can begin.

On the other hand, fortunately, the morphology-poor nature of Chinese blurs the difference between verbs, nouns, and adjectives, forming a closer connection between these predicates and their arguments. This allows the training of a unified model across all types of predicates. Yet another property of Chinese that impacts automatic semantic role labeling is that Chinese has many more verb types than English—at least a factor of four—and so in a similarly sized corpus, the number of instances per verb is much smaller for Chinese, which exacerbates an already critical issue of data sparsity. At the same time, this means that there is less polysemy to deal with, which is a preferred property from a performance perspective. The creation of a specific clustering feature helps overcome this problem to some degree. So, in some sense, although a similar set of features are useful across languages, the specific instantiation of some can differ greatly, and the relative benefit of each varies with language as well. These new features were introduced by Xue [167] to improve the performance of the Chinese semantic role labeling system. Another thing to note about Chinese is that the syntactic parsing performance is inferior

to that of English, and so systems based on shallow syntactic chunking [169] achieved quite competitive results with those based on a full syntactic parse.

In contrast to Chinese, a particular characteristic of Arabic is its morphological richness. What this means in parsing is that there are many more syntactic POS categories for Arabic than there are for English or Chinese—almost an order of magnitude. So far, results reported in the literature on Arabic semantic role labeling systems have not exploited its specific morphologically rich nature [168].

Another notable difference from English is that both Arabic and Chinese have many more implicit (or dropped) subjects. The Penn Treebank marks them with a trace, and they are tagged with an argument in PropBank. Unlike English, Chinese and Arabic require the training of special models to identify dropped subjects before the predicate-argument structure can be fully realized [170].

Robustness across Genre

One possible shortcoming of this and other approaches is that all the parsers are trained on the same Penn Treebank, which, when evaluated on sources other than WSJ, seems to degrade in performance. Carreras and Màrquez [171] show that the drop in performance on test data from the Brown corpus was 10 F-score points lower than the test data from WSJ. When trained and tested on WSJ propositions the syntactic parser's performance is the main source of error, and the classification performance is quite good. However, Pradhan, Ward, and Martin [172] report that when we train the system on WSJ data and test on the Brown propositions, the classification performance and the identification performance are affected to the same degree. This provides some evidence that more lexical semantic features are needed to bridge the performance gap across genres. Zapirain [173] shows that incorporating features

based on selectional preferences provides one way of effecting more lexico-semantic generalization.

4.5.3. Software

Following is a list of software packages available for semantic role labeling.

- **ASSERT** (Automatic Statistical SEmantic Role Tagger)

  [http://www.cemantix.org/assert.html]

  A semantic role labeler trained on the English PropBank data.

- **C-ASSERT** [http://hlt030.cse.ust.hk/research/Pc-assert/]

  An extension of ASSERT for the Chinese language.

- **SwiRL** [http://www.surdeanu.name/mihai/swirl/]

  Another semantic role labeler trained on PropBank data.

- **Shalmaneser** (A *Shal*low Se*man*tic Par*ser*)

  [http://www.coli.uni-saarland.de/projects/salsa/shal/]

  A toolchain for shallow semantic parsing based on the FrameNet data.

4.6. Meaning Representation

We now turn to the third, deeper level of semantic interpretation whose objective is to take natural language input and transform it into an unambiguous representation that a machine can act on. This is the form that would be more likely to be as incomprehensible to humans as it would be comprehensible to machines. We can think of a parallel between programming languages that are much closer to the way humans manipulate information and the low-level machine code that the computer executes. Although compilers and interpreters impose various specific syntactic and semantic restrictions on a program written in a highlevel programming language, no such restrictions

are imposed on the form that natural language can take; whereas precision in artificial languages is necessary to define scope and eliminate ambiguity, natural language relies on the recipient to disambiguate it using context and general world knowledge. Researchers have spent decades figuring out how to interpret and/or encode context and use world knowledge so that we can make machines understand what humans seem to understand so effortlessly. However, there is still a lot of progress to be made, and so far the techniques that have been developed only work within specific domains and problems instead of being scalable to arbitrary domains. This is often termed **deep semantic parsing**, as opposed to shallow semantic parsing that comprises word sense disambiguation and semantic role labeling.

4.6.1. Resources

A number of projects have created representations and resources that have promoted experimentation in this area. Let us look at a few of those resources.

ATIS

Though not quite focused on formal knowledge representation, the Air Travel Information System (ATIS) project [174] is considered one of the first concerted efforts to build systems to transform natural language into a representation that could be used by an end application to make decisions. The task involved a machine to transform a user query in spontaneous speech, using a restricted vocabulary, about flight information. It then formed a representation that was compiled into a SQL query, which was used to extract answers from a flight database. A hierarchical frame representation was used to encode the intermediate semantic information. Figure 4–20 shows a sample user query and the corresponding frame representation. The training corpus includes over 774 scenarios completed by 137 subjects, yielding a total of over

7,300 utterances. All utterances are transcribed, and 2,900 of them have been categorized and annotated with canonical reference answers.[12] Roughly 600 of these have also been treebanked.[13]

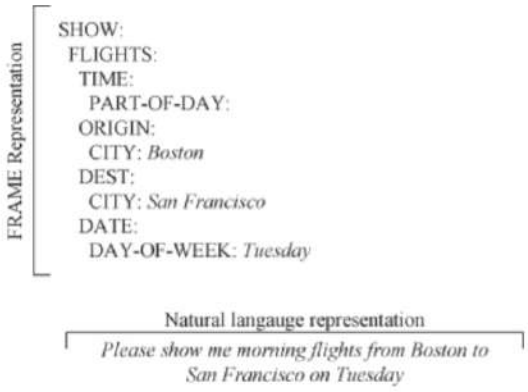

Figure 4–20. A sample user query and its *frame* representation in the ATIS program

Communicator

The Communicator program was the follow-on to ATIS. While ATIS was more focused on **user-initiated dialog** (i.e., the user asked questions to the machine, which provided answers), Communicator involved a **mixed-initiative dialog**, whereby the human and machine had a dialog with each other with the computer presenting users with real-time travel information and helping them negotiate a preferred itinerary. Over the period of the program, many thousands of dialogs were collected and are available through the Linguistic Data Consortium. Carnegie-Mellon University collected more data, a portion of which is available for research.[14] Roughly a million words and ~ 1,600 dialogs have been annotated with dialog acts.[15]

GeoQuery

In the domain of U.S. geography, there is a natural language interface (NLI) to a geographic database called Geobase [175], which has about 800 Prolog facts stored in a relational database with geographic information such as population,

neighboring states, major rivers, and major cities. Some sample queries and their representations are as follows:

1. What is the capital of the state with the largest population?

   answer(C, (capital(S, C), largest(P, (state(S), population(S, P))))

2. What are the major cities in Kansas?

   answer(C, (major(C), city(C), loc(C, S), equal(S, stateid(kansas))))

This is the GeoQuery corpus, which has also been translated into Japanese, Spanish, and Turkish.

Robocup: CLang

RoboCup (www.robocup.org) is an international initiative by the artificial intelligence community that uses robotic soccer as its domain. There is a special formal language, CLang, which is used to encode the advice from the team coach, and the behaviors are expressed as if-then rules. Following is an example representation in this domain:

1. If the ball is in our penalty area, all our players except player 4 should stay in our half.

   ((bpos (penalty-area our)) (do (player-except our 4) (pos (half our))))

4.6.2. Systems

As we can see in these examples, depending on the consuming application, the meaning representation can be a SQL query, a Prolog query, or a domain-specific query representation. Now we look at the various ways the problem of mapping the natural language to such meaning representation has been tackled.

Rule Based

Some of the semantic parsing systems that performed very well for both the ATIS and Communicator projects were rule-based systems in the sense that they used an interpreter whose semantic grammar was handcrafted to be robust to speech recognition errors. The underlying philosophy was that the traditional syntactic explanation of a sentence

is much more complex than the underlying semantic information, so parsing the meaning units in the sentence into semantics proved to be a better approach. Furthermore, especially in dealing with spontaneous speech, the system has to account for ungrammatical instructions, stutters, filled pauses, and so on. Word order therefore becomes less important, which leads to meaning units scattered in the sentences/utterances and not necessarily in the order that would make sense to a syntactic parser. Ward's [176, 177, 178] system, Phoenix, uses recursive transition networks (RTNs) [179] and a handcrafted grammar to extract a hierarchical frame structure, and reevaluates and adjusts the values of these frames with each new piece of information obtained. This system had an error rate of 13.2% for spontaneous speech input with a speech recognition word-error rate of 4.4%, and a 9.3% error for transcript input.

Supervised

Although rule-based techniques are relatively easy to craft in the beginning and serve a good purpose to formulate solutions to various tasks, they have several downsides: (i) they need some effort upfront to create the rules, (ii) the time and specificity required to write rules usually restricts the development to systems that operate in limited domains, (iii) they are hard to maintain and scale up as the problem becomes more complex and more domain independent, and (iv) they tend to be brittle. The alternative is to use statistical models derived from hand-annotated data. However, unless some hand-annotated data is available, statistical models cannot be used to deal with unknown phenomena. During the ATIS evaluations, some data was hand-tagged for semantic information. Schwartz et al. [180] used this as an opportunity to create what was probably the first end-to-end supervised statistical learning system for the ATIS domain. They had four components in their system: (i) semantic parse, (ii) semantic frame, (iii) discourse, and (iv) backend. This system used a supervised learning approach

combined with quick training augmentation through a humanin-the-loop corrective approach to generate slightly lower quality but more data for improved supervision. Miller et al. [181] described the algorithm in more detail. Their system achieved an error rate of 14.5% on the entire test set and 9.5% on the subset of sentences that were context independent. Since then, various improvements have been made, such as by He and Young [182].

Continuing on the line of research that is today commonly known as natural language interface for databases (NLIDB), Zelle and Mooney [183] tackled the task of retrieving answers from a Prolog database by converting natural language questions into Prolog queries in the domain of GeoQuery. They introduced a system called CHILL (Constructive Heuristics Induction for Language Learning), based on the relational learning techniques of inductive logic programming. It uses a shift-reduce parser to map the input sentence into parses expressed as a Prolog program. They preferred a representation closer to formal logic rather than SQL, because once achieved, it can easily be translated into other equivalent representations. They tested the system performance with the rule-based system Geobase that comes with the GeoQuery dataset over a varying number of queries as inputs. It took CHILL roughly 175 training queries to match the performance of Geobase. Additional queries made it surpass Geobase, achieving an accuracy of 84% on novel queries, at times inducing 1,100 lines of Prolog code.

Since then, advances have been made in machine learning and syntactic parsing, and researchers have identified new approaches and refined existing approaches. The SCISSOR (Semantic Composition that Integrates Syntax and Semantics to get Optimal Representation) system, for example, uses a statistical syntactic parser to create a **semantically augmented parse tree** (SAPT) [184, 185]. Training for SCISSOR consists of a (natural language, SAPT, meaning representation) triplet. It uses a standard

syntactic parser augmented with semantic tags, then a recursive procedure is used to compositionally construct the meaning representation for each node in the tree given the representations of its children. SCISSOR shows significant performance improvement over earlier approaches. KRISP (Kernel-based Robust Interpretation for Semantic Parsing) [186] uses string kernels and SVMs to improve the underlying learning techniques. WASP (Word Alignmentbased Semantic Parsing) [187] takes a radical approach to semantic parsing by using state-of-the-art machine translation techniques to learn a semantic parser. Wong and Mooney treat the meaning representation language as an alternative form of natural language and use GIZA++ to produce an alignment between the natural language and a variation of the meaning representation language. Complete meaning representations are then formed by combining these aligned strings using a synchronous CFG (SCFG) framework. SCISSOR is somewhat more accurate than WASP and KRISP, which can themselves benefit from the information in SAPTs [188]. KRISP, CHILL, and WASP have also learned semantic parsers for Spanish, Turkish, and Japanese with similar accuracies. Yet another approach comes from Zettlemoyer and Collins [189], who trained a structured classifier for natural language interfaces by learning probabilistic combinatory categorial grammar (PCCG) along with a log-linear model that represents the distribution over the syntactic and semantic analysis conditioned on the natural language input.

4.6.3. Software

Not a lot of software programs are available for the older, more rule-based systems, but the following are available for download.

- **WASP** [http://www.cs.utexas.edu/~ml/wasp/]
- **KRISPER** [http://www.cs.utexas.edu/~ml/krisp/]
- **CHILL** [http://www.cs.utexas.edu/~ml/chill.html]

word similarity measure based on the categories in Wikipedia.

4.5. Predicate-Argument Structure

Shallow semantic parsing, or what is popularly known today as **semantic role labeling**, is the process of identifying the various arguments of predicates in a sentence. The linguistic community has been debating for a few decades over what constitutes the set of arguments and what the granularity of such argument labels should be for various predicates, which in turn can be the verbs, nouns, adjectives, and prepositions in a sentence [65, 66].

4.5.1. Resources

The late 1990s saw the emergence of two important corpora that are semantically tagged. One is FrameNet[4] [67, 68, 69, 70] and the other is PropBank[5] [71]. These resources have begun a transition from a long tradition of predominantly rule-based approaches toward more data-oriented approaches. These approaches focus on transforming linguistic insights into features, rather than into rules, and letting a machine learning framework use those features to learn a model that helps automatically tag the semantic information encoded in such resources. FrameNet is based on the theory of **frame semantics**, where a given predicate invokes a **semantic frame**, thus instantiating some or all of the possible semantic roles belonging to that frame [72]. PropBank, on the other hand, is based on Dowty's [73] prototype theory and uses a more linguistically neutral view in which each predicate has a set of core arguments that are predicate dependent, and all predicates share a set of noncore, or adjunctive, arguments. It builds on the syntactic Penn Treebank corpus. We now discuss these approaches in more detail.

FrameNet

FrameNet contains frame-specific semantic annotation of

a number of predicates in English. It contains tagged sentences extracted from the British National Corpus (BNC). The process of FrameNet annotation consists of identifying specific semantic frames and creating a set of frame-specific roles called **frame elements**. Then, a set of predicates that instantiate the semantic frame, irrespective of their grammatical category, are identified, and a variety of sentences are labeled for those predicates. The labeling process entails identifying the frame that an instance of the predicate lemma invokes, then identifying semantic arguments for that instance, and tagging them with one of the predetermined set of frame elements for that frame. The combination of the predicate lemma and the frame that its instance invokes is called a lexical unit (LU). This is therefore the pairing of a word with its meaning. Each sense of a polysemous word tends to be associated with a unique frame. For example, the verb *break* can mean *fail to observe (a law, regulation, or agreement)* and can belong to a COMPLIANCE frame along with other word meanings such as *violation, obey, flout*; or it can mean *cause to suddenly separate into pieces in a destructive manner* and can belong to a CAUSE_TO_FRAGMENT frame along with other meanings such as *fracture, fragment, smash*.

The following example illustrates the general idea. Here, the frame AWARENESS is instantiated by the verb predicate *believe* and the noun predicate *comprehension*. Figure 4–9 shows the AWARENESS frame along with the frame-elements and a sample set of predicates that involve it—including verbs and nominalizations.

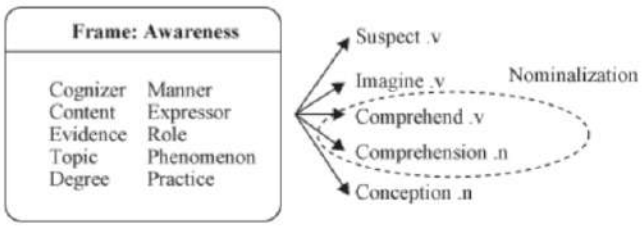

Figure 4–9. FrameNet example

1. [$_{Cognizer}$ We] [$_{Predicate:verb}$ *believe*] [$_{Content}$ it is a fair and generous price]
2. No doubts existed as to [$_{Cognizer}$ our] [$_{Predicate:noun}$ *comprehension*] [$_{Content}$ of it]

FrameNet encompasses a wide variety of nominal predicates. They include ultra-nominals [74, 73], nominals, and nominalizations. FrameNet also contains some adjective and preposition predicates.

To get an idea of the amount of data we are talking about as of this writing, the latest release of FrameNet, R1.5, contains about 173,000 predicate instances covering about 8,000 frame elements from roughly 1,000 frames over the BNC. Although the number of frame elements seems very large, many share the same meaning across the 11,000 lexical units. For example, the frame element BODY_PART in frame CURE has the same meaning as the same element in the frame GESTURE or in WEARING.

PropBank

PropBank only includes annotations of arguments of verb predicates. All the noncopular verbs in the WSJ part of the Penn Treebank [75] have been labeled with their semantic arguments. PropBank restricts the argument boundaries to that of a syntactic constituent, as defined in the Penn Treebank. It uses a linguistically neutral terminology to label the arguments. The arguments are tagged as either **core arguments**, with labels of the type ARGN, where N takes values from 0 to 5, or **adjunctive arguments** (listed in Table 4–2), with labels of the type ARGM-X, where *X* can take values such as TMP for temporal, LOC for locative, and so on. Adjunctive arguments share the same meaning across all predicates, whereas the meaning of core arguments has to be interpreted in connection with a predicate. ARG0 is the PROTO-AGENT (usually the subject of a transitive verb), ARG1 is the PROTO-PATIENT (usually its direct object of the transitive verb) [73]. Table 4–1 shows a list of core arguments

for the predicates *operate* and *author*. Note that some core arguments, such as ARG2 and ARG3, do not occur with *author*. This is explained by the fact that not all core arguments can be instantiated by all senses of all predicates. A list of core arguments that can occur with a particular sense of the predicate, along with their real-world meaning, is present in a file called the *frames* file. One frames file is associated with each predicate.

Table 4–1. Argument labels associated with the predicate *operate.01* (sense: work), and for *author.01* (sense: to write or construct) in the PropBank corpus

Predicate	Argument	Description
operate.01		
	ARG0	Agent, operator
	ARG1	Thing operated
	ARG2	Explicit patient (thing operated on)
	ARG3	Explicit argument
	ARG4	Explicit instrument
author.01		
	ARG0	Author, agent
	ARG1	Text authored

Table 4–2. List of adjunctive arguments in PropBank—ARGMs

Tag	Description	Examples
ARGM-LOC	Locative	*the museum, in Westborough, Mass.*
ARGM-TMP	Temporal	*now, by next summer*
ARGM-MNR	Manner	*heavily, clearly, at a rapid rate*
ARGM-DIR	Direction	*to market, to Bangkok*
ARGM-CAU	Cause	*In response to the ruling*
ARGM-DIS	Discourse	*for example, in part, Similarly*
ARGM-EXT	Extent	*at $38.375, 50 points*
ARGM-PRP	Purpose	*to pay for the plant*
ARGM-NEG	Negation	*not, n't*
ARGM-MOD	Modal	*can, might, should, will*
ARGM-REC	Reciprocals	*each other*
ARGM-PRD	Secondary Predication	*to become a teacher*
ARGM	Bare ARGM	*with a police escort*
ARGM-ADV	Adverbials	*(none of the above)*

An example extracted from the PropBank corpus along with its syntax tree representation and argument labels is shown in Figure 4–10.

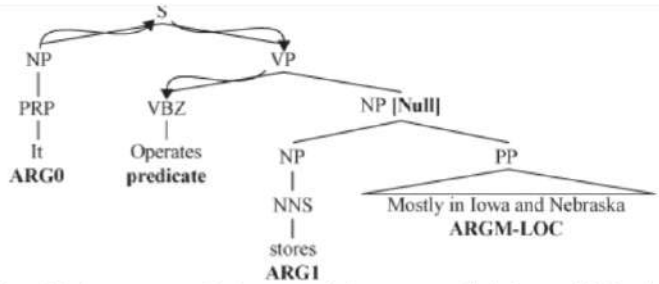

Figure 4–10. Syntax tree for a sentence illustrating the PropBank tags

Most Treebank-style trees have **trace nodes** that refer to another node in the tree but have no words associated with them. These can also be marked as arguments. Because traces are not reproduced by a usual syntactic parser, the community has disregarded them from most standard experiments. PropBank also contains arguments that are coreferential. As with any strategy that integrates multiple layers of annotation, there exist some disagreements between the Treebank and PropBank annotation. Sometimes

the PropBankers strongly believed an error existed in the tree or the tree structure was not amenable to a one-to-one mapping between the argument and a tree node. In such cases, they annotated a sequence of nodes in the tree as the argument. These are called **discontiguous arguments**, and very few (~ 1%–2%) cases exist in the data. There are around 115,000 predicate instances instantiating about 250,000 instances of the roughly 20 argument types over about 5,000 frames in the WSJ portion of the PropBank release. There are about 18,000 more predicates annotated with arguments from the Brown Corpus. More recently, the OntoNotes project [22, 23, 24] has annotated more corpora from various genres with predicate-argument structures using the PropBank guidelines. This has led to modifications to the Penn Treebank and PropBank guidelines to generate a better, more aligned resource [76]. Most experiments discussed in this chapter use the WSJ annotation from PropBank v1.0.

An important distinction to note between the FrameNet and PropBank corpora is that in FrameNet there exist lexical units, which are words paired with their meanings or the frames that they invoke, whereas in PropBank each lemma has a list of different **framesets** that represent all the senses for which there is a different argument structure. These are akin to word senses but tend to be coarser grained [77].

Other Resources

Other resources have been developed to aid further research in predicate-argument recognition. NomBank [78] was inspired by PropBank. In the process of identifying and tagging the arguments of nouns, the NOMLEX (NOMinalization LEXicon) [79] dictionary was expanded to cover about 6,000 entries. Along with this, the frames from PropBank were used to generate the frame files for NomBank. Another resource that ties PropBank frames with more predicate-independent thematic roles and also provides a richer representation associating the framesets with Levin classes [80] is VerbNet [81]. In fact the PropBank frames have

a strong connection with the Levin verb classes, specifically the intersective Levin classes [82]. FrameNet frames are also somewhat related in the sense that FrameNet's generation of verb classes is more data driven than theoretical. Baker and Ruppenhofer [83] present an interesting discussion on how the FrameNet frames relate to Levin classes.

Although FrameNet and PropBank started with annotating predicate-argument structures in English, it was not long before their philosophy propagated to other languages. Because FrameNet was based on frame semantics with coarse-grained semantic frames, and the nature of semantics is lingua independent, it was apparent that these frames could be reused to annotate data in other languages. The SALSA project [84, 85] was the first to put this into practice. FrameNet tags both literal and metaphorical interpretations of text, but that can lead to ambiguity and lower consistency, so the SALSA project remained close to the literal meaning. It reused all possible, preexisting FrameNet frames, and when linguistic semantic parallelism did not propagate across languages, they created more frames. Subsequently, there exist FrameNets in Japanese [86, 87], Spanish [88], and Swedish [89]. As of this writing, there are FrameNet projects underway in more than 10 languages [90].

PropBank has inspired the creation of similar resources in Chinese [91], Arabic [92, 93], Korean [94], Spanish, Catalan [95], and most recently, Hindi [96]. Many of these involve the same core researchers. Unlike FrameNet, every new PropBank requires the creation of a new set of frame files.

Although the FrameNet and PropBank philosophies have inspired similar projects in other languages, they are not the only styles used in practice. For example, the Prague Dependency Treebank [97] takes a different approach, tagging the predicate-argument structure in its tectogrammatic layer on top of the dependency structure. It also makes a distinction similar to that of core and adjunctive arguments called **inner participants** and **free**

modifications. The NAIST Text Corpus [98] is strongly influenced by the traditions in Japanese linguistics.

4.5.2. Systems

Unlike word sense disambiguation, little research has gone into learning predicate-argument structures from unannotated corpora, perhaps because it is closer to the actual applications and has been more or less absorbed in the area of information extraction. Most early systems, such as KL-ONE [99] and others [100, 101], were based primarily on heuristics on syntax trees, which were in turn rule based until the Penn Treebank was available as a training resource for supervised syntactic parsers. Most of these systems dealt with (predicate-independent) thematic roles. There has been a great deal of linguistic inquiry into the nature of argument structure, but most of it did not directly apply to domainindependent understanding systems. Until corpora were available, the main resources were rules based on syntactic trees. One significantly useful resource, mentioned earlier in context of PropBank, was the classification of verbs and their alternations by Levin [80]. The Absity parser [102, 13] is one of the first rule-based semantic parsers. Also notable was the parser used in the PUNDIT understanding system [103, 104]. Efforts were later made to use a hybrid method for thematic role tagging [105, 106] using WordNet as a resource for domain-specific interpretation [107]. Other notable efforts are the corpus-based studies by Manning [108] and Briscoe and Carroll [109], which seek to derive the subcategorization information from large corpora, and by Pustejovsky [110], which tries to acquire lexical semantic knowledge from corpora.

A major leap forward in semantic role labeling research happened after the introduction of FrameNet and PropBank. One big problem with the FrameNet philosophy, and also partly with that of PropBank, is that significant work goes into creating the frames, that is, in classifying verbs into the

framesets in preparation for manual annotation. Therefore, providing coverage for all possible verbs in one or more languages requires significant manual effort upfront. Green, Dorr, and Resnik [111] propose a way to learn the frame structures automatically, but the result is not accurate enough to replace the manual frame creation. Swier and Stevenson [112] represent one of the more recent approaches to handling this problem in an unsupervised fashion.

Let us now review the more recent and common approaches since the advent of these corpora. The process of semantic role labeling can be defined as identifying a set of word sequences, each of which represents a semantic argument of a given predicate. For example, for the sentence in Figure 4–10, for the predicate *operates*, the word *I* fills the role ARG0, the word *stores* fills the role ARG1, and the sequence of words *mostly in Iowa and Nebraska* fills the role ARGM-LOC. Recall that PropBank is largely agnostic to any generalizations made across predicates; an ARGN for one predicate need not have similar semantics compared to another predicate.[6]

FrameNet was the first project that used hand-tagged arguments of predicates in data, and Gildea and Jurafsky [113] were the first to formulate semantic role labeling as a supervised classification problem that assumes the arguments of a predicate and the predicate itself can be mapped to a node in the syntax tree for that sentence. They introduced three tasks that could be used to evaluate the system and that have become standard since:

Argument identification—This is the task of identifying all and only the parse constituents that represent valid semantic arguments of a predicate.

Argument classification—Given constituents known to represent arguments of a predicate, assign the appropriate argument labels to them.

Argument identification and classification—This task is a combination of the previous two tasks, where the

constituents that represent arguments of a predicate are identified, and the appropriate argument label is assigned to them.

Once the sentence has been parsed using a syntactic parser, each node in the parse tree can be classified as either one that represents a semantic argument (i.e., non-null node) or one that does not represent any semantic argument (i.e., a null node). The non-null nodes can then be further classified into the set of argument labels.

For example, in the tree of Figure 4–10, the noun phrase that encompasses *stores mostly in Iowa and Nebraska* is a null node because it does not correspond to a semantic argument. The node NP that encompasses *stores* is a non-null node because it does correspond to a semantic argument: ARG1.

The pseudocode for a generic semantic role labeling (SRL) algorithm is shown in Algorithm 4–3.

Syntactic Representations

As we have seen, PropBank was created as a layer of annotation on top of Penn Treebank– style phrase structure trees, and in some of the early work in recovering PropBank annotations, Gildea and Jurafsky [113] added argument labels to parses obtained from a parser trained on Penn Treebank. In subsequent years, researchers have used various other types of sentence representations, either directly or as an independent source of evidence, to tackle the semantic role labeling problem. We look at each of these sentence representations in turn and at the features that were used to tag text with PropBank arguments.

Algorithm 4–3. The semantic role labeling (SRL) algorithm

Procedure: SRL(sentence) **returns** best *semantic role labeling*

Input: *syntactic parse* of the *sentence*

1: *generate* a full syntactic parse of the *sentence*
2: *identify* all the *predicates*
3: **for all** *predicate* ∈ *sentence* **do**
4: *extract* a set of features for each node in the tree relative to the *predicate*
5: *classify* each feature vector using the *model* created in training
6: *select* the class of highest scoring classifier
7: **return** best *semantic role labeling*
8: **end for**

Phrase Structure Grammar (PSG) FrameNet marks word spans in sentences to represent arguments, whereas PropBank tags nodes in a treebank tree with arguments. Because several high-quality statistical parsers existed that could produce phrase structure trees, and because the phrase structure representation is amenable to tagging, Gildea and Jurafsky [113] used it. They introduced the following features, some of which were extracted from the parse tree of the sentence:

Path—This feature is the syntactic path through the parse tree from the parse constituent to the predicate being classified. For example, in Figure 4–10, the path from ARG0 *It* to the predicate *operates* is represented by the string NP ↑ S ↓ VP ↓ VBZ. ↑ and ↓ represent upward and downward movement in the tree respectively.

Predicate—The identity of the predicate lemma is used as a feature.

Phrase type—This feature is the syntactic category (NP, PP, S, etc.) of the constituent to be labeled.

Position—This feature is a binary feature identifying whether the phrase is before or after the predicate.

Voice—This feature indicates whether the predicate is realized as an active or passive construction. A set of

handwritten tgrep2[7] expressions on the syntax tree are used to identify the passive-voiced predicates.

Head word—This feature is the syntactic head of the phrase. It is calculated using a head word table described by Magerman [114] and modified by Collins [115].

Subcategorization—This feature is the phrase structure rule expanding the predicate's parent node in the parse tree. For example, in Figure 4–10, the subcategorization for the predicate *operates* is VP → VBZ-NP.

Verb clustering—The predicate is one of the most salient features in predicting the argument class. Given various syntactic/semantic constructions that a predicate can appear in, any amount of hand-tagged training data would be relatively limited for estimating the parameters of a model, and any real-world test set will contain predicate sense/frames that have not been seen in training. In these cases, researchers can benefit from some information about the predicate by creating clusters or classes and using them as features. Gildea and Jurafsky [113] used a distance function for clustering that is based on the intuition that verbs with similar semantics will tend to have similar direct objects. For example, verbs such as *eat*, *devour*, and *savor* will tend to all occur with direct objects describing food. The clustering algorithm uses a database of verb–direct object relations extracted by Lin [116]. The verbs were clustered into 64 classes using the probabilistic co-occurrence model of Hofmann and Puzicha [117].

Surdeanu et al. [118] suggested the following additional features:

Content word—Because the head word feature of some constituents, such as PP and SBAR, is not very informative, they defined a set of heuristics for some constituent type. A different set of rules was used to

identify a so-called **content** word instead of using the usual head word–finding rules. This was used as an additional feature. The rules that they used are shown in Figure 4–11.

```
H1: if phrase type is PP then select the rightmost child
    Example: phrase = "in Texas," content word = "Texas"
H2: if phrase type is SBAR then select the leftmost sentence (S*) clause
    Example: phrase = "that occurred yesterday," content word = "occurred"
H3: if phrase type is VP then
      if there is a VP child then
        select the leftmost VP child
      else
        select the head word
    Example: phrase = "had placed," content word = "placed"
H4: if phrase type is ADVP then select the rightmost child, not IN or TO
    Example: phrase = "more than," content word = "more"
H5: if phrase type is ADJP then select the rightmost adjective, verb, noun, or ADJP
    Example: phrase = "61 years old," content word = "61"
H6: for all other phrase types select the head word
    Example: phrase = "red house," content word = "red"
```

Figure 4–11. List of content word heuristics

Part of speech of the head word and content word—Adding part of speech of the head word and content word of a constituent as a feature to help generalize in the task of argument identification gave a significant performance boost to their decision tree-based system.

Named entity of the content word—Certain roles, such as ARGM-TMP and ARGM-LOC, tend to contain TIME or PLACE named entities. This information was added as a set of binary-valued features.

Boolean named entity flags—Surdeanu et al. also added named entity information as a feature. They created indicator functions for each of the seven named entity types: PERSON, PLACE, TIME, DATE, MONEY, PERCENT, ORGANIZATION.

Phrasal verb collocations—This feature comprises frequency statistics related to the verb and the immediately following preposition.

Fleischman, Kwon, and Hovy [119] added the following features to their system:

Logical function—This is a feature that takes three values—external argument, object argument, and other argument—and is computed using some heuristics on the syntax tree.

Order of frame elements—This feature represents the position of a frame element relative to other frame elements in a sentence.

Syntactic pattern—This feature is also generated using heuristics on the phrase type and the logical function of the constituent.

Previous role—This is a set of features indicating the n^{th} previous role that had been observed/assigned by the system for the current predicate.

Pradhan et al. [120] suggested using the following additional feature variations:

Named entities in constituents—Surdeanu et al. [118] reported a performance improvement on classifying the semantic role of the constituent by using the presence of a named entity in the constituent. Some of these entities, such as location and time, are particularly important for the adjunctive arguments ARGM-LOC and ARGM-TMP. Entity tags should also help in cases where the head words are not common or for a closed set of locative or temporal cues, such as *in Mexico*, or *in 2003*. They also tagged seven named entities in the corpus using IdentiFinder [121] and added them via seven binary features. Each of these features is true if its respective type of named entity is contained in the constituent.

Verb sense information—The arguments that a predicate can take depend on the sense of the predicate. Each predicate tagged in the PropBank corpus is assigned a separate set of arguments depending on the sense in which it is used. This is also known as the **frameset ID**. Table 4–3 illustrates the argument sets for

a word. Depending on the sense of the predicate *talk*, either ARG1 or ARG2 can identify the *hearer*. Absence of this information can be potentially confusing to the learning mechanism.

Table 4–3. Argument labels associated with the two senses of predicate *talk* in PropBank corpus

talk.01		talk.02	
Tag	Description	Tag	Description
ARG0	Talker	ARG0	Talker
ARG1	Subject	ARG1	Talked to
ARG2	Hearer	ARG2	Secondary action

Verb sense information extracted from PropBank is added by treating each sense of a predicate as a distinct predicate, which helps performance. The disambiguation of PropBank framesets can be performed at a very high accuracy [122].

Noun head of prepositional phrases—Many adjunctive arguments, such as temporals and locatives, occur as prepositional phrases in a sentence, and the head words of those phrases, which are always prepositions, often are not very discriminative. For instance, *in the city* and *in a few minutes* both share the same head word *in*, and neither contains a named entity, but the former is ARGM-LOC, whereas the latter is ARGM-TMP. Therefore, Pradhan et al. [120] replaced the head word of a prepositional phrase with that of the first noun phrase inside the prepositional phrase. The preposition information was retained by appending it to the phrase type; for example, the head word of the prepositional phrase *for about 20 minutes* was originally the preposition *for*. After the transformation, the head word was changed to *minutes*, and the phrase PP was changed to PP-FOR. The head word was used in its surface form as well as a lemmatized form. The lemmatization was performed automatically using the XTAG morphology database [123].[8]

First and last word/POS in constituent—Some arguments tend to contain discriminative first and last words, so these were used along with their part of speech as four new features.

Ordinal constituent position—This feature avoids false positives where constituents far away from the predicate are spuriously identified as arguments. It is a concatenation of the constituent type and its ordinal position from the predicate.

Constituent tree distance—This is a finer way of specifying the already present position feature, where the distance of the constituent from the predicate is measured in terms of the number of nodes that need to be traversed through the syntax tree to go from one to the other.

Constituent relative features—These are nine features representing the constituent type, head word and head word part of speech of the parent, and left and right siblings of the constituent in focus. These were added on the intuition that encoding the tree context this way might add robustness and improve generalization.

Temporal cue words—Several temporal cue words are not captured by the named entity tagger and were therefore added as binary features indicating their presence.

The BOW (Bag of words) toolkit[9] was used to identify words and bigrams that had highest average mutual information with the ARGM-TMP argument class.

Dynamic class context—In the task of argument classification, these are dynamic features that represent the hypotheses of, at most, the previous two non-null nodes belonging to the same tree as the node being classified.

Path generalizations—As will be seen in Section 4.5.2, for the argument identification task, the path is one of

the most salient features. However, it is also the most datasparse feature. To overcome this problem, the path was generalized in several different ways:

Clause-based path variations—Position of the clause node (S, SBAR) seems to be an important feature in argument identification [124]. Accordingly, Pradhan et al. [120] experimented with four clause-based path feature variations.

- Replacing all the nodes in a path other than clause nodes with an (*). For example, the path NP ↑ S ↑ VP ↑ SBAR ↑ NP ↑ VP ↓ VBD becomes NP ↑ S ↑ *S ↑ * ↑ * ↓ VBD. SBAR is replaced with S.
- Retaining only the clause nodes in the path, which for the above example would produce NP ↑ S ↑ S ↓ VBD.
- Adding a binary feature that indicates whether the constituent is in the same clause as the predicate,
- Collapsing the nodes between S nodes, which gives NP ↑ S ↑ NP ↑ VP ↓ VBD.

Path *n*-grams—This feature decomposes a path into a series of trigrams. For example, the path NP ↑ S ↑ VP ↑ SBAR ↑ NP ↑ VP ↓ VBD becomes NP ↑ S ↑ VP, S ↑ VP ↑ SBAR, VP ↑ SBAR ↑ NP, SBAR ↑ NP ↑ VP, and so on. Shorter paths were padded with nulls.

Single-character phrase tags—Each phrase category is clustered to a category defined by the first character of the phrase label.

Path compression—Compressing sequences of identical labels into following the intuition that successive embedding of the same phrase in the tree might not add additional information.

Directionless path—Removing the direction in the path, thus making insignificant the point at which it changes direction in the tree.

Partial path—Using only that part of the path from the constituent to the lowest common ancestor of the predicate and the constituent. For example, the partial path for the path illustrated in Figure 4–10 is NP ↑ S.

Another work that deals with paths in an orthogonal fashion is that of Vickrey and Koller [125]. They perform a rule-based sentence simplification in an attempt to automatically accrue path generalizations.

Predicate context—This feature captures predicate sense variations. Two words before and two words after were added as features. The part of speech of the words were also added as features.

Punctuations—For some adjunctive arguments, punctuation plays an important role. This set of features captures whether punctuation appears immediately before and after the constituent.

Feature context—Features of constituents that are parent or siblings of the constituent being classified were found useful. Traditionally, each constituent is classified independently; however, in reality, there is a complex interaction between the types and number of arguments that a constituent can have. In other words, the classification of each argument is dependent on the classifications of other nodes. As we will see later, the method of Pradhan et al. performs a postprocessing step using the argument sequence information, but that does not cover all possible constraints. One way of trying to capture those best in the current architecture would be to take into consideration the feature vector compositions of all the non-null constituents for the sentence. This is exactly what this feature does. It uses

all the other feature vector values of the constituents that are likely to be non-null, as an added context.

Combinatory Categorial Grammar (CCG) As we learned, although the path feature is very important for the argument identification task, it is one of the most sparse features and may be difficult to train or generalize [126, 127]. A dependency grammar should generate shorter paths from the predicate to dependent words in the sentence and could be a more robust complement to the phrase structure grammar paths extracted from the PSG parse tree. Gildea and Hockenmaier [128] report that using features extracted from a CCG representation improves semantic role labeling performance on core arguments (ARG0-5). Because CCG trees are binary trees and the constituents have poor alignment with the semantic arguments of a predicate, the researchers performed experiments using head words rather than the entire span. Later, Pradhan et al. (2005) [129] used these features to augment the original phrase structure tree-based algorithm to accrue further benefits.

Figure 4–12 shows the CCG parse of the sentence *London denied plans on Monday*. Gildea and Hockenmaier [128] introduced three features:

Phrase type—This is the category of the maximal projection between the two words, the predicate and the dependent word.

Categorial path—This is a feature formed by concatenating the following three values: (i) category to which the dependent word belongs, (ii) the direction of dependence, and (iii) the slot in the category filled by the dependent word. For example, for the tree in Figure 4–12, the path between *denied* and *plans* would be (S[dcl]\NP)/NP.2.←.

Tree Path—This is the categorial analogue of the path feature in the Charniak parsebased system, which traces

the path from the dependent word to the predicate through the binary CCG tree.

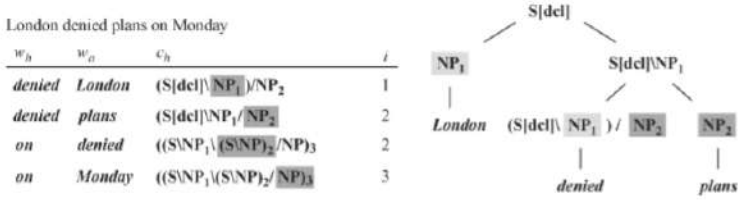

Figure 4–12. Combinatory categorial grammar parse

Tree-Adjoining Grammar (TAG) Chen and Rambow [130] report results using two different sets of features: (i) surface syntactic features much like the Gildea and Palmer [131] system and (ii) additional features that result from the extraction of a TAG from the Penn Treebank. They chose a TAG because of its ability to address long-distance dependencies in text. The additional features they introduced are:

Supertag path—This feature is the same as the path feature seen earlier except that in this case it is derived from a TAG rather than from a PSG.

Supertag—This feature is the tree frame corresponding to the predicate or the argument.

Surface syntactic role—This feature is the surface syntactic role of the argument.

Surface subcategorization—This feature is the subcategorization frame.

Deep syntactic role—This feature is the deep syntactic role of an argument, whose values include *subject* and *direct object*.

Deep subcategorization—This is the deep syntactic subcategorization frame. For example, for a transitive verb, it would be NP0 NP1. When available, the preposition modifying the NP is used to lexicalize the feature. So, in case of the predicate *load*, a possible frame would be NP0 NP1 NP2(into).